

Aufgabenstellung

Entwickeln Sie in einer Gruppe eine vollständige Java-Anwendung zu einem realitätsnahen Verwaltungsthema.

Die Anwendung soll aus einem **fachlichen Modell**, einer **Verwaltungsschicht**, **Fehlerbehandlung mit Exceptions**, **Dateiverarbeitung** sowie einer **grafischen Benutzeroberfläche mit Vaadin** bestehen.

Die Software muss so aufgebaut sein, dass sie nicht nur Daten speichert, sondern auch fachliche Logik ausführt, Eingaben prüft, Daten dauerhaft sichert und über eine GUI bedienbar ist.

Die Umsetzung erfolgt in Gruppen.

Rahmenbedingungen

- Gruppenarbeit mit **1 bis 3 Personen**
- Umsetzung als vollständiges Softwareprojekt
- Verwendung von **Java** und **Vaadin** verpflichtend

Ziel

Entwickeln Sie eine **komplexere, realitätsnahe Verwaltungssoftware**, die folgende Aspekte kombiniert:

- objektorientierte Modellierung
- Vererbung & Polymorphie
- komplexe Datenverarbeitung
- Fehlerbehandlung mit eigenen Exceptions
- Persistenz (Dateien)
- Testen
- mehrschichtige Architektur
- moderne GUI mit Vaadin (mehrere Views)

Eigene Themen sind erlaubt, sofern sie ausreichend komplex sind.

Anforderungen

1. Klassenmodell

Das Projekt muss ein strukturiertes Klassenmodell enthalten:

- 1 abstrakte Superklasse
- mindestens 3 Subklassen
- eine Verwaltungsklasse
- eventuell weitere Hilfsklassen (z. B. für Validierung oder Dateioperationen)

Anforderungen an die Superklasse

- mindestens **4 Attribute**
- Konstruktor(en)
- Getter und Setter
- mindestens **2 Methoden mit Logik**
- mindestens **2 abstrakte Methoden**

Anforderungen an die Subklassen

- jeweils mindestens **2 zusätzliche Attribute**
- eigene Implementierungen der abstrakten Methoden
- zusätzliche eigene Methoden mit fachlicher Bedeutung

Weitere Anforderungen

- Verwendung von **Vererbung und Polymorphie**

2. Verwaltungsklasse

Die Verwaltungsklasse ist für die Organisation aller Objekte zuständig.

Anforderungen

- Speicherung der Objekte in **Listen**
- optional zusätzlich: Nutzung von Maps

Pflichtfunktionen

- Objekte hinzufügen
- Objekte entfernen
- Objekte suchen
- alle Objekte anzeigen

- Objekte nach verschiedenen Kriterien filtern
- Sortierung nach mindestens zwei verschiedenen Kriterien
- Gruppierung von Objekten (z. B. nach Klasse)

Auswertungen

Die Anwendung muss mehrere Berechnungen enthalten, z. B.:

- Durchschnittswerte
- Summen
- Minimum / Maximum
- statistische Auswertungen

Mindestens **eine Methode muss mehrere Verarbeitungsschritte kombinieren** (z. B. filtern und berechnen oder filtern und sortieren).

3. Exception-Behandlung

Die Anwendung muss ein eigenes Exception-Konzept verwenden.

Anforderungen

- **eine eigene Exception-Klasse**
 - Validierung von Eingaben in:
 - Konstruktoren
 - Settern
 - Methoden
 - Verwendung von **try-catch** in der Verwaltung
 - Fehlerbehandlung in der GUI
-

4. Dateioperationen

Die Anwendung muss Daten speichern und laden können.

Anforderungen

- Speichern in Datei (z. B. CSV oder Textformat)
- Laden von Daten beim Start der Anwendung
- separate Methoden für die Dateioperationen

- Behandlung von Ein-/Ausgabe-Fehlern
-

5. Testen

Erstellen Sie eine JUnit-Testklasse zum Testen der Verwaltungsklasse.

6. Benutzeroberfläche mit Vaadin

Die Anwendung muss eine grafische Benutzeroberfläche besitzen.

Aufbau

- mindestens **3 Views (Seiten)**
- Navigation zwischen den Views

Pflichtfunktionen

- Eingabeformular für neue Objekte
- Bearbeiten bestehender Objekte
- Löschen von Objekten
- Anzeige aller Daten in einer Tabelle (Grid)
- Suchfunktion
- Filterfunktion mit mehreren Kriterien
- Sortierung über die Oberfläche steuerbar
- Anzeige von Auswertungen

Anforderungen an die GUI

- klare Struktur und gute Bedienbarkeit
 - Verwendung typischer Vaadin-Komponenten
 - Anzeige von Fehlermeldungen bei ungültigen Eingaben
-

7. Architektur

Die Anwendung muss klar strukturiert sein:

- **Modell (Klassen)**
- **Verwaltungsklasse mit Fachlogik**
- **Benutzeroberfläche (GUI) mit Event Handlern**

Die GUI darf keine Fachlogik enthalten, sondern greift auf die Fachlogik der Verwaltungsklasse zu.

8. UML-Diagramm

Erstellen Sie ein Klassendiagramm mit:

- allen Klassen
 - Attributen und Methoden
 - Sichtbarkeiten
 - Vererbungsbeziehungen
 - Beziehungen zwischen Klassen
-

9. Dokumentation

Die Dokumentation soll enthalten:

- Beschreibung des Projekts
 - Erklärung der Klassenstruktur
 - Begründung der Vererbung
 - Beschreibung der wichtigsten Methoden
 - Erklärung der Dateioperationen
 - Beschreibung der GUI-Struktur
 - Aufteilung der Arbeit in der Gruppe
-

10. Fachliche Funktionen

Die Anwendung muss mindestens **8 bis 10 fachliche Funktionen** enthalten, z. B.:

- komplexe Filter
- kombinierte Suche und Sortierung
- Berechnungen und Statistiken
- Zustandsänderungen
- Vergleich von Objekten
- Grenzwertüberprüfungen

11. Zusätzliche Funktionen

Mindestens **2 der folgenden Funktionen** müssen zusätzlich umgesetzt werden:

- Verwendung mehrerer Listen
- Verlaufs- oder Logfunktion
- Markierungssystem (z. B. Favoriten)
- Import/Export von Daten
- einfache Benutzerrollen (z. B. Admin/User)

Mindestumfang

Das Projekt soll mindestens enthalten:

- 6–8 Klassen
- 1 abstrakte Superklasse
- 3 Subklassen
- 1 Verwaltungsklasse
- 1 JUnit-Testklasse
- eine Exception-Klasse
- Dateioperationen
- mehrere Views in Vaadin
- umfangreiche Fachlogik

Abgabe

Abzugeben sind:

- vollständiges Java-Projekt
- Vaadin-Anwendung
- UML-Diagramm
- Dokumentation

Abgabegespräch:

- Präsentation
 - Vorführung der Anwendung
-

Bewertungskriterien

- Qualität des Klassenmodells
- sinnvolle Verwendung von Vererbung
- Umfang und Qualität der Funktionen
- Struktur und Lesbarkeit des Codes
- korrekte Verwendung von Exceptions
- funktionierende Dateioperationen
- Testfälle
- Gestaltung und Funktionalität der GUI
- Vollständigkeit der Umsetzung